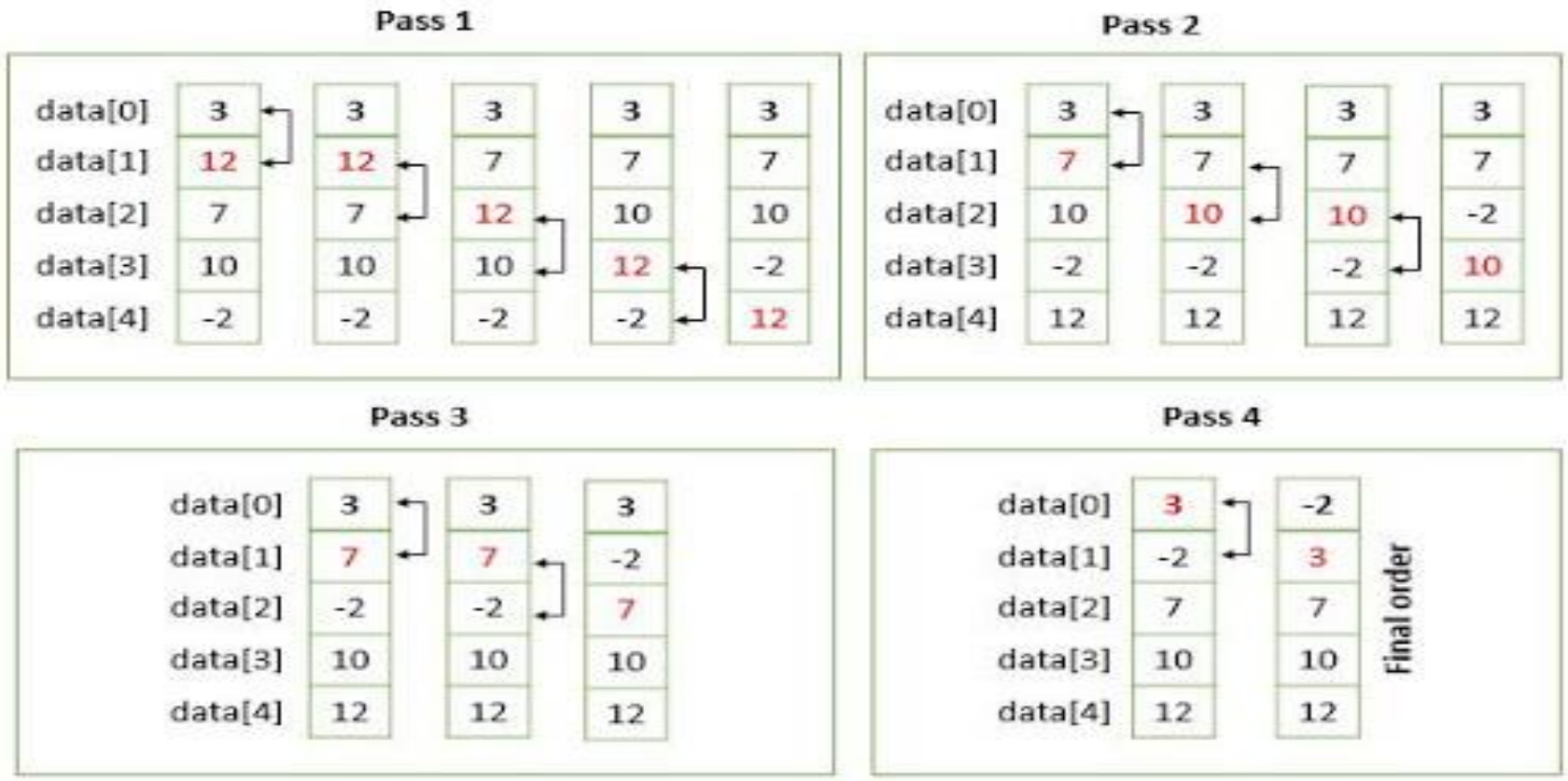
Sorting

Arranging the items in ascending or descending order

* **Sorting can be performed using several techniques or methods, as follows:**

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Heap Sort
6. Merge sort
7. Radix sort

Bubble Sort (Example 1)



Algorithm

bubble_sort(A[ ], n)

Here A[] is an array of n elements. This algorithm sorts the elements of A[] in ascending order by using bubble sort process.

1. Repeat step 2 and 3 for k = 1 to n-1 // control the passes
2. Repeat step 3 for i = 1 to n-k // control comparison of two successive elements
3. if A[i] > A[i+1] then
   // interchange
   temp = A[i]
   A[i] = A[i+1]
   A[i + 1] = temp

[End of inner loop(step 2)]

[End of outer loop (step 1)]

4. Exit

Bubble sort Algorithm

Algorithm for sorting an array (Bubble sort)

(Bubble sort) BUBBLE (DATA, N)

Here DATA is an array with N elements. This algorithm sorts the elements in DATA.

1. Repeat Steps 2 and 3 for K = 1 to N – 1.
2. Set PTR := 1. [Initialize pass pointer PTR.]
3. Repeat while PTR <= (N – K): [Executes pass.]
   (a) If DATA[PTR] > DATA[PTR + 1], then:
   Interchange DATA[PTR] and DATA[PTR + 1].
   [End of If structure.]
   (b) Set PTR := PTR + 1.
   [End of inner loop.]
   [End of Step 1 outer loop.]
4. Exit.

Bubble Sort (Example 3)

Suppose the following numbers are stored in an array A:

32, 51, 27, 85, 66, 23, 13, 57

We apply the bubble sort to the array A. We discuss each pass separately.

Pass 1. We have the following comparisons:

(a) Compare A_1 and A_2. Since $32 < 51$, the list is not altered.

(b) Compare A_2 and A_3. Since $51 > 27$, interchange 51 and 27 as follows:

32, (27), (51), 85, 66, 23, 13, 57

(c) Compare A_3 and A_4. Since $51 < 85$, the list is not altered.

(d) Compare A_4 and A_5. Since $85 > 66$, interchange 85 and 86 as follows:

32, 27, 51, (66), (85), 23, 13, 57

(e) Compare A_5 and A_6. Since $85 > 23$, interchange 85 and 23 as follows:

32, 27, 51, 66, (23), (85), 13, 57

(f) Compare A_6 and A_7. Since $85 > 13$, interchange 85 and 13 to yield:

32, 27, 51, 66, 23, (13), (85), 57

(g) Compare A_7 and A_8. Since $85 > 57$, interchange 85 and 51 to yield:

32, 27, 51, 66, 23, 13, (57), (85)

Bubble Sort ..

At the end of this first pass, the largest number, 85, has moved to the last position. However, the rest of the numbers are not sorted, even though some of them have changed their positions.

For the remainder of the passes, we show only the interchanges.

Pass 2. (27, 33,) 51, 66, 23, 13, 57, 85

27, 33, 51, (23, 66,) 13, 57, 85

27, 33, 51, 23, (13, 66,) 57, 85

27, 33, 51, 23, 13, (57, 66,) 85

At the end of Pass 2, the second largest number, 66, has moved its way down to the next-to-last position.

Pass 3. 27, 33, (23, 51,) 13, 57, 66, 85

27, 33, 23, (13, 51,) 57, 66, 85

Pass 4. 27, (23, 33,) 13, 51; 57, 66, 85

27, 23, (13, 33,) 51, 57, 66, 85

Pass 5. (23, 27,) 13, 33, 51, 57, 66, 85

23, (13, 27,) 33, 51, 57, 66, 85

Pass 6. (13, 23,) 27, 33, 51, 57, 66, 85

Pass 6 actually has two comparisons, A_1 with A_2 and A_2 and A_3. The second comparison does not involve an interchange.

Pass 7. Finally, A_1 is compared with A_2. Since $13 < 23$, no interchange takes place.

Since the list has 8 elements; it is sorted after the seventh pass. (Observe that in this example, the list was actually sorted after the sixth pass. This condition is discussed at the end of the section.)

Bubble Sort (Example 4 Character type Data)

Using the bubble sort algorithm, Algorithm 4.4, find the number C of comparisons and the number D of interchanges which alphabetize the $n = 6$ letters in PEOPLE.

The sequences of pairs of letters which are compared in each of the $n - 1 = 5$ passes follow: a square indicates that the pair of letters is compared and interchanged, and a circle indicates that the pair of letters is compared but not interchanged.

Pass 1. [P E] O P L E, E [P O] P L E, E O (P P) L E
E O P [P L] E E O P L [P E] E O P L E P

Pass 2. (E O) P L E P, E (O P) L E P, E O [P L] E P
E O L [P E] P, E O L E P P

Pass 3. (E O) L E P P, E [O L] E P P, E L [O E] P P
E L E O P P

Pass 4. (E L) E O P P, E [L E] O P P, E E L O P P

Pass 5. (E E) L O P P, E E L O P P

Since $n = 6$, the number of comparisons will be $C = 5 + 4 + 3 + 2 + 1 = 15$. The number D of interchanges depends also on the data, as well as on the number n of elements. In this case $D = 9$.

Bubble Sort (Example 5 String Type)

```
   pen   sad   cat   ant   pan   bed   can   but
Pass 1:
   pen   sad   cat   ant   pan   bed   can   but
   pen   cat   sad   ant   pan   bed   can   but
   pen   cat   ant   sad   pan   bed   can   but
   pen   cat   ant   pan   sad   bed   can   but
   pen   cat   ant   pan   bed   sad   can   but
   pen   cat   ant   pan   bed   can   sad   but
   pen   cat   ant   pan   bed   can   but   sad

Pass 2:
   cat   pen   ant   pan   bed   can   but   sad
   cat   ant   pen   pan   bed   can   but   sad
   cat   ant   pan   pen   bed   can   but   sad
   cat   ant   pan   bed   pen   can   but   sad
   cat   ant   pan   bed   can   pen   but   sad
   cat   ant   pan   bed   can   but   pen   sad

Pass 3:
   ant   cat   pan   bed   can   but   pen   sad
   ant   cat   pan   bed   can   but   pen   sad
   ant   cat   bed   pan   can   but   pen   sad
   ant   cat   bed   can   pan   but   pen   sad
   ant   cat   bed   can   but   pan   pen   sad
```

Bubble Sort (Example 5 ..String Type)

```
Pass 4:
   ant   cat   bed   can   but   pan   pen   sad
   ant   bed   cat   can   but   pan   pen   sad
   ant   bed   can   cat   but   pan   pen   sad
   ant   bed   can   but   cat   pan   pen   sad

Pass 5:
   ant   bed   can   but   cat   pan   pen   sad
   ant   bed   can   but   cat   pan   pen   sad
   ant   bed   but   can   cat   pan   pen   sad

Pass 6:
   ant   bed   but   can   cat   pan   pen   sad
   ant   bed   but   can   cat   pan   pen   sad

Pass 7:
   ant   bed   but   can   cat   pan   pen   sad
```

Bubble Sort (Example 6 String Type)

0	1	2	3	4	5	6	7
was	had	him	and	you	his	the	but
had	was	him	and	you	his	the	but
had	him	was	and	you	his	the	but

and but had him his the was you

Time Complexity

* Time complexity is a type of computational complexity that describes the time required to execute an algorithm. The time complexity of an algorithm is the amount of time it takes for each statement to complete.

* Big O notation is a mathematical notation used in computer science to describe the upper bound or worst-case scenario of the runtime complexity of an algorithm in terms of the input size.

* Big-O, commonly referred to as “Order of”, is a way to express the upper bound of an algorithm’s time complexity, since it analyses the worst-case situation of algorithm. It provides an upper limit on the time taken by an algorithm in terms of the size of the input. It’s denoted as O(f(n)), where f(n) is a function that represents the number of operations (steps) that an algorithm performs to solve a problem of size n.

* Complexity Comparison Between Typical Big Os
* O(1) - Constant Time Complexity
* Description: Algorithms with constant time complexity execute in a constant amount of time regardless of the input size.
* Example: Accessing an element in an array by index.
* Comparison: Regardless of the input size, the time is the same.
* O(log n) - Logarithmic Time Complexity
* Description: Algorithms with logarithmic time complexity have their runtime grow logarithmically with the input size.
* Example: Binary search in a sorted array.
* Comparison: As the input size increases, the runtime grows slowly, making it more efficient than linear time complexities.
* O(n) - Linear Time Complexity
* Description: Algorithms with linear time complexity have their runtime grow linearly with the input size.
* Example: Linear search through an unsorted array.
* Comparison: The runtime increases proportionally to the input size.
* O(n log n) - Linearithmic Time Complexity
* Description: Algorithms with linearithmic time complexity have their runtime grow in proportion to the input size multiplied by the logarithm of the input size.
* Example: Efficient sorting algorithms like mergesort and heapsort.
* Comparison: More efficient than quadratic time complexities but less efficient than linear or logarithmic ones.

* O(n^2) - Quadratic Time Complexity
* Description: Algorithms with quadratic time complexity have their runtime grow quadratically with the input size.
* Example: Nested loops iterating over the input.
* Comparison: As the input size increases, the runtime grows quadratically, making it less efficient for large inputs.
* O(2^n) - Exponential Time Complexity
* Description: Algorithms with exponential time complexity have their runtime grow exponentially with the input size.
* Example: Brute-force algorithms that try all possible combinations.
* Comparison: Extremely inefficient for large inputs, as the runtime increases rapidly with even small increases in input size.
* O(n!) - Factorial Time Complexity
* Description: Algorithms with factorial time complexity have their runtime grow factorially with the input size.
* Example: Algorithms generating all permutations of a set.
* Comparison: Highly inefficient, with the runtime growing extremely fast with the input size.

Bubble Sort Time Complexity

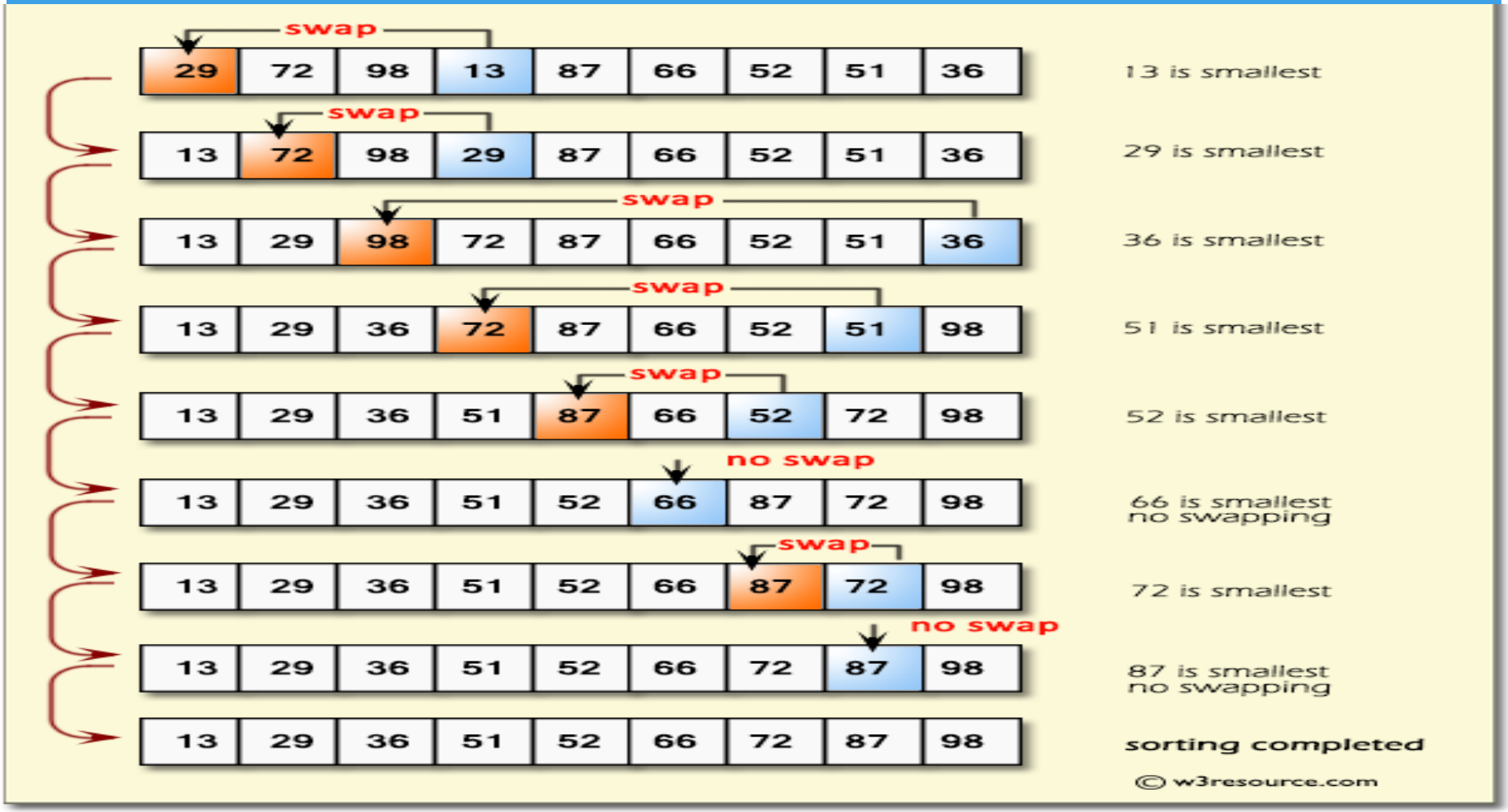
- **Best-Case Time Complexity**
  - Array is already sorted
  - (N-1) + (N-2) + (N-3) + …. + (1) = (N-1)* N / 2 comparisons

Called Linear Time
$O(N^2)$
Order-of-N-square

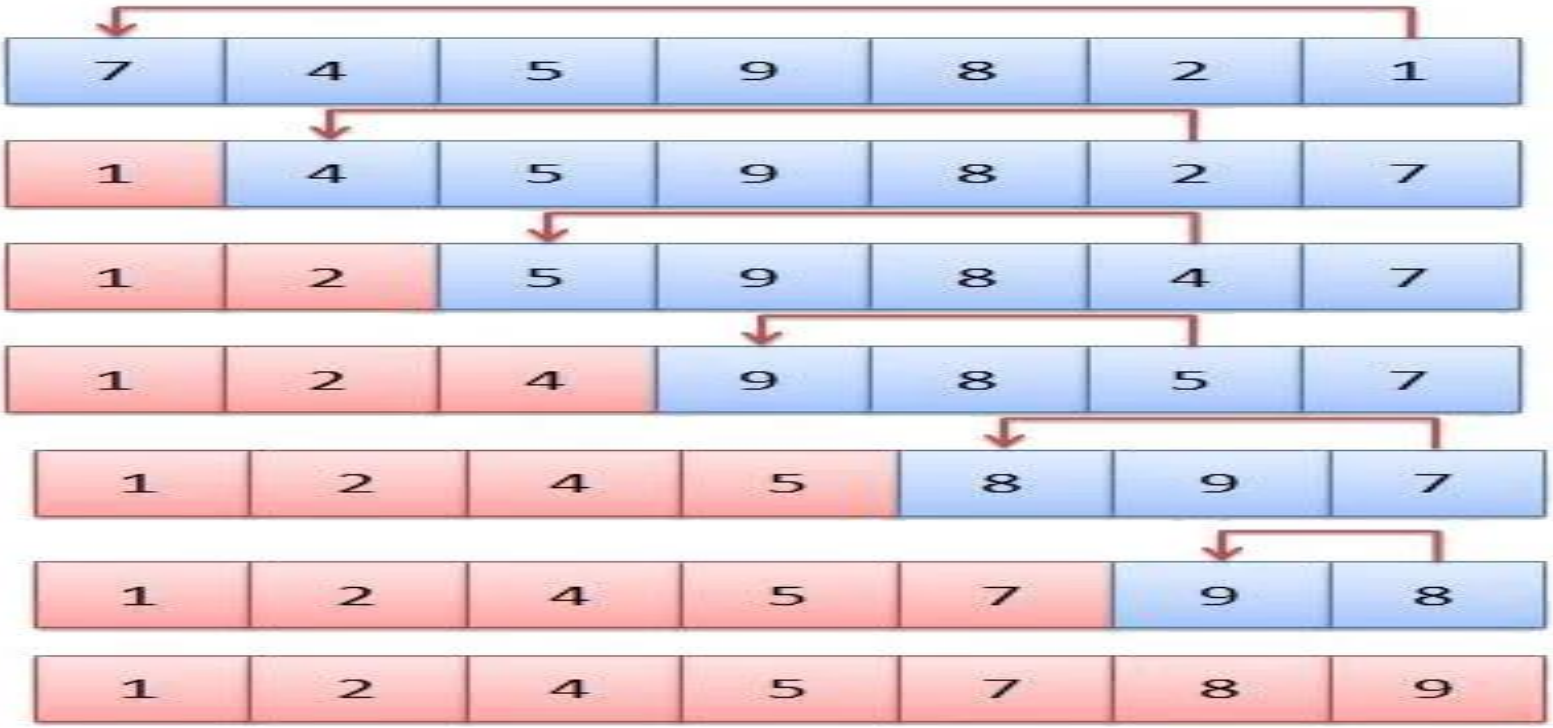
- **Worst-Case Time Complexity**
  - Need N-1 iterations
  - (N-1) + (N-2) + (N-3) + …. + (1) = (N-1)* N / 2

Called Quadratic Time
$O(N^2)$
Order-of-N-square

Selection Sort (Example 1)



Selection Sort (Example 2)



Selection Sort

This algorithm sorts an array A with N elements.

SELECTION(A, N)

1. Repeat steps 2 and 3 for k=1 to N-1:
2. Call MIN(A, K, N, LOC).
3. [Interchange A[k] and A[LOC]]
   Set Temp:= A[k], A[k]:= A[LOC] and A[LOC]:=Temp.
   [End of step 1 Loop.]
4. Exit.

MIN(A, K, N, LOC).

1. Set MIN := A[K] and LOC:= K.
2. Repeat for j=k+1 to N:
   If Min>A[j], then: Set Min:= A[j] and LOC:=J.
   [End of if structure]
3. Return.

Selection Sort Complexity

The number f(n) of comparisons in selection sort algorithm is independent of original order of elements. There are n-1 comparisons during pass 1 to find the smallest element, n-2 comparisons during pass 2 to find the second smallest element, and so on.

Accordingly,

$$f(n) = (n-1)+(n-2)+\text{-----}+2+1$$
$$= n(n-1)/2$$
$$= O(n^2)$$

The f (n) holds the same value $O(n^2)$ both for worst case and average case.

Selection Sort

* https://www.google.com/url?sa=i&url=https%3A%2F%2Falgorithms.tutorialhorizon.com%2Fselection-sort-java-implementation%2Fselection-sort-gif%2F&psig=AOvVaw14Y1_PwPH4TJrYugN9HOSL&ust=1588174718002000&source=images&cd=vfe&ved=0CAIQjRxqFwoTCLDU0ou6i-kCFQAAAAAdAAAAABAn

Radix Sort (Example 1)

Suppose 9 cards are punched as follows:

348, 143, 361, 423, 538, 128, 321, 543, 366

Given to a card sorter, the numbers would be sorted in three phases, as pictured in Fig. 9.6:

Input	0	1	2	3	4	5	6	7	8	9
348										
143										
361		361		143					348	
423										
538				423						
128										
321									538	
543		321							128	
366				543						
							366			

(a) First pass

Radix Sort..

Input	0	1	2	3	4	5	6	7	8	9
361							361			
321			321							
143					143					
423			423							
543					543					
366					543					
366							366			
348					348					
538				538						
128			128							

(b) Second pass

Radix Sort..

Input	0	1	2	3	4	5	6	7	8	9
321				321						
423					423					
128		128								
538						538				
143		143								
543						543				
348				348						
361				361						
366				366						

(c) Third pass

128, 143, 321, 348, 361, 366, 423, 538, 543

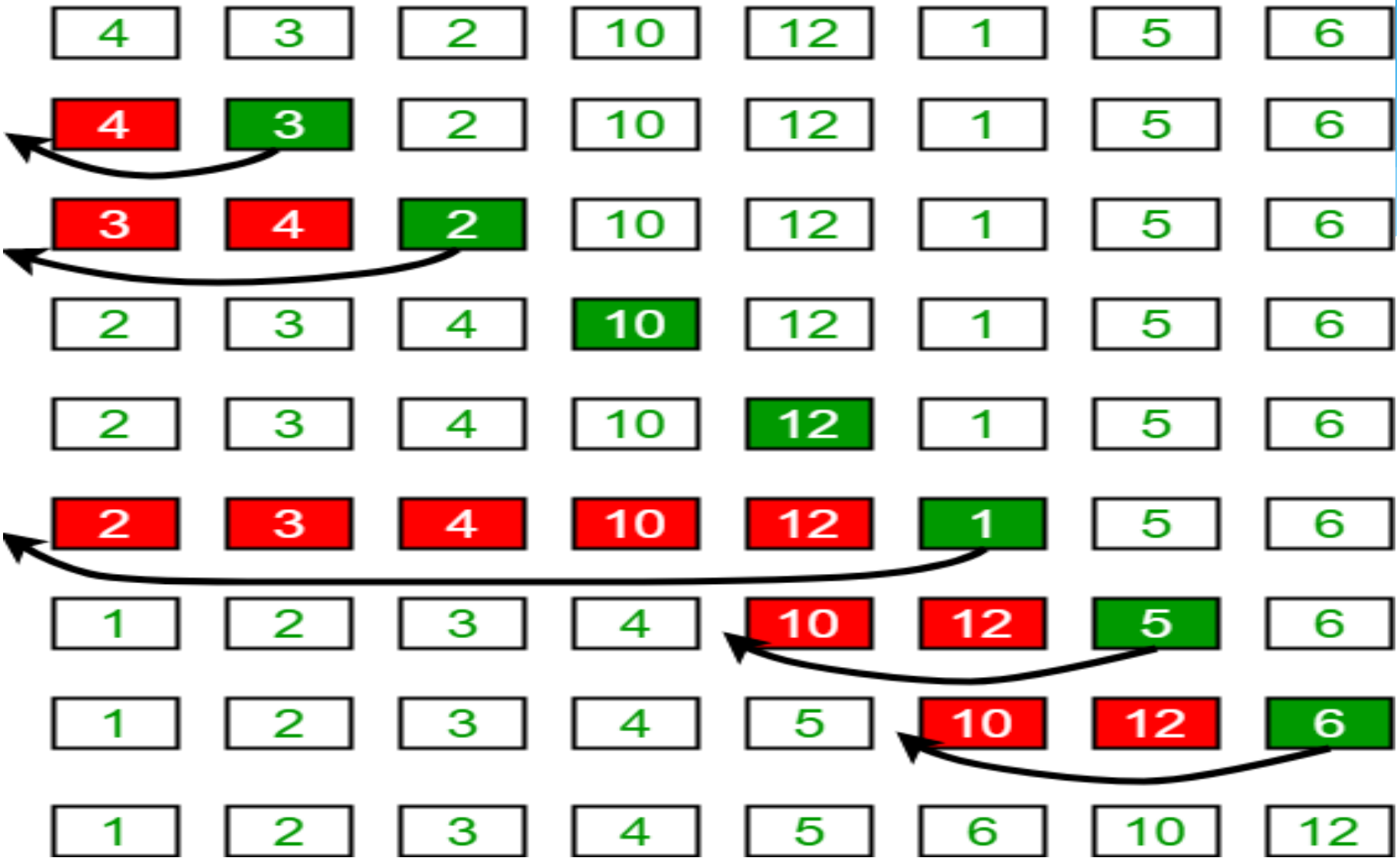
Radix Sort (Example 2)

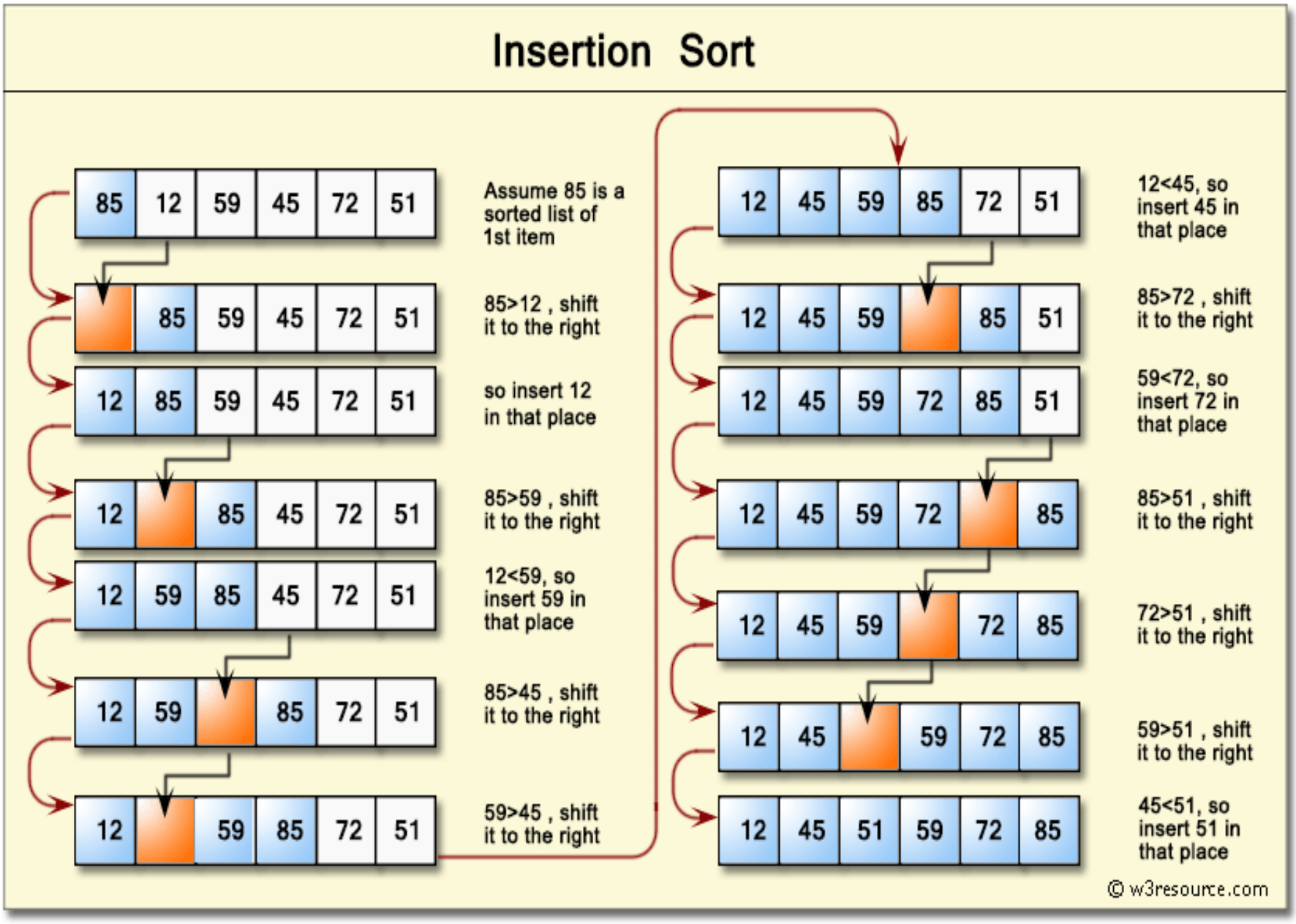
36 9 0 25 1 49 64 16 81 4

Bin	0	1	2	3	4	5	6	7	8	9
Content	0	1 81	-	-	64 4	25	36 16	-	-	9 49

Bin	0	1	2	3	4	5	6	7	8	9
Content	0 1 4 9	16	25	36	49	-	64	-	81	-

Insertion Sort Execution Example




Insertion Sort
85 12 59 45 72 51
Assume 85 is a sorted list of 1st item
85 59 45 72 51
85>12 , shift it to the right
12 85 59 45 72 51
so insert 12 in that place
12 85 45 72 51
85>59 , shift it to the right
12 59 85 45 72 51
12<59, so insert 59 in that place
12 59 85 72 51
85>45 , shift it to the right
12 59 85 72 51
59>45 , shift it to the right
12 45 59 85 72 51
12<45, so insert 45 in that place
12 45 59 85 51
85>72 , shift it to the right
12 45 59 72 85 51
59<72, so insert 72 in that place
12 45 59 72 85
85>51 , shift it to the right
12 45 59 72 85
72>51 , shift it to the right
12 45 59 72 85
59>51 , shift it to the right
12 45 51 59 72 85
45<51, so insert 51 in that place
© w3resource.com

9.6 MERGE-SORT

Suppose an array A with n elements A[1], A[2], ..., A[N] is in memory. The merge-sort algorithm which sorts A will first be described by means of a specific example.

Example 9.7

Suppose the array A contains 14 elements as follows:

66, 33, 40, 22, 55, 88, 60, 11, 80, 20, 50, 44, 77, 30

Each pass of the merge-sort algorithm will start at the beginning of the array A and merge pairs of sorted subarrays as follows:

Pass 1. Merge each pair of elements to obtain the following list of sorted pairs:

33, 66 22, 40 55, 88 11, 60 20, 80 44, 50 30, 70

Pass 2. Merge each pair of pairs to obtain the following list of sorted quadruplets:

22, 33, 40, 66 11, 55, 60, 88 20, 44, 50, 80 30, 77

Pass 3. Merge each pair of sorted quadruplets to obtain the following two sorted subarrays:

11, 22, 33, 40, 55, 60, 66, 88 20, 30, 44, 50, 77, 80

Pass 4. Merge the two sorted subarrays to obtain the single sorted array

11, 20, 22, 30, 33, 40, 44, 50, 55, 60, 66, 77, 80, 88

The original array A is now sorted.

Algorithm	Average Case	Worst Case
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Bubble Sort	$O(n^2)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$
Radix Sort	O(nk)	O(nk)

Time Complexity of Typical Sorting Algorithms.